

Sector 0 — Carga / Descarga de tablas

Grupo A — Numpy · Pandas · CSV

0.1A) Carga de CSV con pandas

```
# [SINTAXIS GENÉRICA] pd.read_csv carga un archivo tabular a DataFrame
# Parametrizaciones clave: index_col (columna como índice), sep (delimitador),
# header (fila de cabecera), encoding, na_values (valores a interpretar como NaN)
df = pd.read_csv("ruta/archivo.csv") # carga básica, índice 0..N
df = pd.read_csv("ruta/archivo.csv", index_col="id_col") # usa columna como índice
df = pd.read_csv("ruta/archivo.csv", sep=";", encoding="utf-8") # CSV europeo con separador ;

# [EJEMPLO APLICADO] EuInfo.csv – 39 países europeos, 9 columnas
eu = pd.read_csv("EuInfo.csv") # shape (39, 9), índice 0..38
eu = pd.read_csv("EuInfo.csv", index_col="Código") # índice: AD, AL, AT, BA...
```

0.2A) Descarga a CSV

```
# [SINTAXIS GENÉRICA] to_csv exporta DataFrame a archivo plano
# index=False evita escribir el índice como columna adicional
# mode="a" permite append (concatenar al archivo existente)
df.to_csv("ruta/salida.csv", index=False) # exporta sin índice
df.to_csv("ruta/salida.csv", index=True) # exporta con índice como 1ª columna
df.to_csv("ruta/salida.csv", mode="a", header=False) # append sin reescribir cabecera

# [EJEMPLO APLICADO] Exportar subset de países con región definida
eu.dropna(subset=["Regions"]).to_csv("EuInfo_con_region.csv", index=False)
```

0.3A) Parametrización del índice (reset, set, drop)

```
# [SINTAXIS GENÉRICA] Control del índice: reset_index vuelve el índice a columna
# drop=True descarta el índice viejo; set_index fija un índice explícito
df = df.reset_index() # índice viejo -> nueva columna "index"
df = df.reset_index(drop=True) # descarta índice viejo, reenumera 0..N
df = df.set_index("col_clave") # fija columna como índice permanente
df.index.name = "identificador" # renombra el nombre del índice

# [EJEMPLO APLICADO] Manipulación de índice en EuInfo
eu_idx = eu.set_index("Código") # índice por código de país
eu_reset = eu_idx.reset_index() # vuelve "Código" a columna normal
eu_flat = eu.reset_index(drop=True) # índice limpio 0..38
```

```
# [SINTAXIS GENÉRICA] Control del índice: reset_index vuelve el índice a columna
# drop=True descarta el índice viejo; set_index fija un índice explícito
df = df.reset_index() # índice viejo -> nueva columna "index"
df = df.reset_index(drop=True) # descarta índice viejo, reenumera 0..N
df = df.set_index("col_clave") # fija columna como índice permanente
df.index.name = "identificador" # renombra el nombre del índice

# [EJEMPLO APLICADO] Manipulación de índice en EuInfo
eu_idx = eu.set_index("Código") # índice por código de país
eu_reset = eu_idx.reset_index() # vuelve "Código" a columna normal
eu_flat = eu.reset_index(drop=True) # índice limpio 0..38
```

 https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html · https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html · https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.reset_index.html · https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.set_index.html

Grupo B — PostgreSQL · SQLAlchemy · DuckDB · CSV

0.1B) Conectar el engine (SQLAlchemy)

Python

```
# [SINTAXIS GENÉRICA] create_engine inicializa un pool de conexiones a la base de datos
# El string de conexión sigue el formato: dialecto+driver://usuario:pass@host:puerto/bd
# echo=True muestra el SQL generado en consola (útil para debug)
from sqlalchemy import create_engine
engine = create_engine("postgresql+psycopg2://user:pass@localhost:5432/midb", echo=False)

# [EJEMPLO APLICADO] Conexión a PostgreSQL local con base de datos "eurodata"
engine = create_engine("postgresql+psycopg2://admin:secret@localhost:5432/eurodata")
conn = engine.connect() # conexión explícita para transacciones manuales
```

0.2B) Inspección de tablas en PostgreSQL (SQLAlchemy + SQL nativo)

Python

```
# [SINTAXIS GENÉRICA] inspect() permite obtener metadatos sin cargar datos
# get_columns devuelve nombre, tipo y nullable de cada columna
from sqlalchemy import inspect, text
insp = inspect(engine)
insp.get_columns("nombre_tabla") # lista de dicts con metadatos de columnas
insp.get_table_names() # lista de tablas visibles en el schema

# [EJEMPLO APLICADO] Listar tablas y columnas de la base eurodata
insp = inspect(engine)
print(insp.get_table_names()) # ['países', 'renta_hogares', ...]
print(insp.get_columns("países")) # [{'name': 'codigo', 'type': 'VARCHAR'}, ...]
```

0.3B) Inspección de CSV en crudo con DuckDB (sin cargar en pandas)

sql

```
-- [SINTAXIS GENÉRICA] DuckDB lee CSV directamente con funciones nativas
-- read_csv_auto infiere tipos, headers y delimitador automáticamente
-- DESCRIBE muestra schema; SUMMARIZE muestra estadísticas descriptivas
DESCRIBE SELECT * FROM read_csv_auto('ruta/archivo.csv');
SUMMARIZE SELECT * FROM read_csv_auto('ruta/archivo.csv');
SELECT * FROM read_csv_auto('ruta/archivo.csv') LIMIT 5;

-- [EJEMPLO APLICADO] EuInfo.csv y RentaDisponibleBrutaHogares.csv
DESCRIBE SELECT * FROM read_csv_auto('EuInfo.csv');
SUMMARIZE SELECT * FROM read_csv_auto('RentaDisponibleBrutaHogares.csv');
SELECT COUNT(DISTINCT geo) AS n_paises FROM read_csv_auto('RentaDisponibleBrutaHogares.csv'); -- 34 geografías
```

 [SQLAlchemy Engine](#) · [SQLAlchemy inspect](#) · [DuckDB CSV import](#) · [DuckDB SUMMARIZE](#)

Sector 1 — Variables y Primer Filtro (First Filter)

Grupo A — Numpy · Pandas · CSV

1.1A) Tipos de variable: dict, Series, ndarray, DataFrame

```
# [SINTAXIS GENÉRICA] Creación y conversión entre estructuras de datos
# dict -> DataFrame (orient controla la estructura del diccionario resultante)
# Series.values extrae el array numpy subyacente (zero-copy cuando es posible)
s = pd.Series([10, 20, 30], index=["a", "b", "c"])      # Serie etiquetada 1D
df = pd.DataFrame({"col1": [1,2], "col2": [3,4]})      # DataFrame 2D tabular
arr = s.values                                         # ndarray de numpy (vista)
d = df.to_dict(orient="list")                         # DataFrame -> dict de listas
d_rec = df.to_dict(orient="records")                  # DataFrame -> lista de dicts (filas)
```

```
# [EJEMPLO APLICADO] Extraer arrays y convertir desde EuInfo
codigos = eu["Código"]                               # Series de 39 elementos
arr_codigos = codigos.values                         # ndarray de strings
dict_eu = eu[["Código", "Nombre del país"]].to_dict(orient="list")
print(type(arr_codigos), arr_codigos.shape)          # <class 'numpy.ndarray'> (39,)
```

```
# [SINTAXIS GENÉRICA] Creación y conversión entre estructuras de datos
# dict -> DataFrame (orient controla la estructura del diccionario resultante)
# Series.values extrae el array numpy subyacente (zero-copy cuando es posible)
s = pd.Series([10, 20, 30], index=["a", "b", "c"])      # Serie etiquetada 1D
df = pd.DataFrame({"col1": [1,2], "col2": [3,4]})      # DataFrame 2D tabular
arr = s.values                                         # ndarray de numpy (vista)
d = df.to_dict(orient="list")                         # DataFrame -> dict de listas
d_rec = df.to_dict(orient="records")                  # DataFrame -> lista de dicts (filas)
```

```
# [EJEMPLO APLICADO] Extraer arrays y convertir desde EuInfo
codigos = eu["Código"]                               # Series de 39 elementos
arr_codigos = codigos.values                         # ndarray de strings
dict_eu = eu[["Código", "Nombre del país"]].to_dict(orient="list")
print(type(arr_codigos), arr_codigos.shape)          # <class 'numpy.ndarray'> (39,)
```

1.2A) dtype, shape y conversión entre tipos

```
# [SINTAXIS GENÉRICA] Inspección de metadatos estructurales
# dtype indica el tipo de dato; shape devuelve tupla de dimensiones
# astype fuerza conversión de tipo (cuidado con pérdida de precisión)
arr = np.array([1.5, 2.5, 3.5])                     # ndarray float64
print(arr.dtype, arr.shape)                          # float64 (3,)
arr_int = arr.astype(np.int32)                       # cast a entero: [1, 2, 3]

print(df.dtypes)                                     # tipo de cada columna
print(df.shape)                                      # (filas, columnas)
df["col"] = df["col"].astype("category")              # conversión a categórica (ahorro memoria)
```

```
# [EJEMPLO APLICADO] Metadatos de los datasets
renta_arr = renta["OBS_VALUE"].values                # ndarray int64 de 379 elementos
print(renta_arr.dtype, renta_arr.shape)              # int64 (379,)
print(eu.dtypes)                                     # object para strings
print(renta.shape)                                   # (379, 12)
```

```
# [SINTAXIS GENÉRICA] Inspección de metadatos estructurales
# dtype indica el tipo de dato; shape devuelve tupla de dimensiones
# astype fuerza conversión de tipo (cuidado con pérdida de precisión)
arr = np.array([1.5, 2.5, 3.5])                     # ndarray float64
print(arr.dtype, arr.shape)                          # float64 (3,)
arr_int = arr.astype(np.int32)                       # cast a entero: [1, 2, 3]

print(df.dtypes)                                     # tipo de cada columna
print(df.shape)                                      # (filas, columnas)
df["col"] = df["col"].astype("category")              # conversión a categórica (ahorro memoria)
```

```
# [EJEMPLO APLICADO] Metadatos de los datasets
renta_arr = renta["OBS_VALUE"].values                # ndarray int64 de 379 elementos
print(renta_arr.dtype, renta_arr.shape)              # int64 (379,)
print(eu.dtypes)                                     # object para strings
print(renta.shape)                                   # (379, 12)
```

Grupo B — PostgreSQL · SQLAlchemy · DuckDB · CSV

1.1B) Consultas básicas de exploración (DuckDB sobre CSV)

```
-- [SINTAXIS GENÉRICA] Primer vistazo a los datos sin cargarlos en memoria local
-- LIMIT restringe filas; DISTINCT elimina duplicados; COUNT(*) cuenta registros
SELECT * FROM read_csv_auto('archivo.csv') LIMIT 5;
SELECT DISTINCT col_categorica FROM read_csv_auto('archivo.csv');
SELECT COUNT(*) AS total_registros FROM read_csv_auto('archivo.csv');
```

```
-- [EJEMPLO APLICADO] Exploración rápida de ambos CSVs
SELECT "Código", "Nombre del país" FROM read_csv_auto('EuInfo.csv') LIMIT 5;
SELECT DISTINCT geo FROM read_csv_auto('RentaDisponibleBrutaHogares.csv');
SELECT MIN(TIME_PERIOD) AS inicio, MAX(TIME_PERIOD) AS fin FROM read_csv_auto('RentaDisponibleBrutaHogares.csv'); --
2014, 2025
```

1.2B) First filter: seleccionar columnas/filas al leer, cargar en DataFrame

```
# [SINTAXIS GENÉRICA] DuckDB permite filtrar y proyectar ANTES de cargar en pandas
# Esto ahorra memoria y tiempo: solo llega al notebook lo estrictamente necesario
# .df() convierte el resultado de la query directamente a pandas DataFrame
import duckdb
con = duckdb.connect()
df_filtrado = con.execute("""
    SELECT col1, col2, col3
    FROM read_csv_auto('archivo.csv')
    WHERE col_condicion IS NOT NULL
""").df()
```

```
# [EJEMPLO APLICADO] Cargar solo países con región definida y renombrar columnas
first_filter = con.execute("""
    SELECT "Código" AS code, "Nombre del país" AS pais, "Regions" AS regiones
    FROM read_csv_auto('EuInfo.csv')
    WHERE "Regions" IS NOT NULL
""").df()
# Resultado: 38 filas x 3 columnas (Belarús tiene Regions=NULL)
```

```
# [SINTAXIS GENÉRICA] DuckDB permite filtrar y proyectar ANTES de cargar en pandas
# Esto ahorra memoria y tiempo: solo llega al notebook lo estrictamente necesario
# .df() convierte el resultado de la query directamente a pandas DataFrame
import duckdb
con = duckdb.connect()
df_filtrado = con.execute("""
    SELECT col1, col2, col3
    FROM read_csv_auto('archivo.csv')
    WHERE col_condicion IS NOT NULL
""").df()
```

```
# [EJEMPLO APLICADO] Cargar solo países con región definida y renombrar columnas
first_filter = con.execute("""
    SELECT "Código" AS code, "Nombre del país" AS pais, "Regions" AS regiones
    FROM read_csv_auto('EuInfo.csv')
    WHERE "Regions" IS NOT NULL
""").df()
# Resultado: 38 filas x 3 columnas (Belarús tiene Regions=NULL)
```

1.3B) First filter secuencial → diccionario de datasets

```
# [SINTAXIS GENÉRICA] Automatizar la carga de múltiples vistas filtradas
# Útil para pipelines donde cada subset requiere transformaciones diferentes
datasets = {}
datasets["vista_1"] = con.execute("SELECT * FROM read_csv_auto('a.csv') WHERE x > 0").df()
datasets["vista_2"] = con.execute("SELECT * FROM read_csv_auto('b.csv') WHERE y = 2025").df()
```

```
# [EJEMPLO APLICADO] Diccionario con datasets de países y renta reciente
datasets = {}
datasets["países_con_region"] = con.execute(
    "SELECT * FROM read_csv_auto('EuInfo.csv') WHERE \"Regions\" IS NOT NULL"
).df()
datasets["renta_2025"] = con.execute(
    "SELECT geo, OBS_VALUE FROM read_csv_auto('RentaDisponibleBrutaHogares.csv') WHERE TIME_PERIOD = 2025"
).df()
```

```
# datasets["países_con_region"].shape = (38, 9)
# datasets["renta_2025"].shape = (10, 2)  -- 10 geografías con datos 2025
```

```
# [SINTAXIS GENÉRICA] Automatizar la carga de múltiples vistas filtradas
# Útil para pipelines donde cada subset requiere transformaciones diferentes
datasets = {}
datasets["vista_1"] = con.execute("SELECT * FROM read_csv_auto('a.csv') WHERE x > 0").df()
datasets["vista_2"] = con.execute("SELECT * FROM read_csv_auto('b.csv') WHERE y = 2025").df()

# [EJEMPLO APLICADO] Diccionario con datasets de países y renta reciente
datasets = {}
datasets["países_con_region"] = con.execute(
    "SELECT * FROM read_csv_auto('EuInfo.csv') WHERE \"Regions\" IS NOT NULL"
).df()
datasets["renta_2025"] = con.execute(
    "SELECT geo, OBS_VALUE FROM read_csv_auto('RentaDisponibleBrutaHogares.csv') WHERE TIME_PERIOD = 2025"
).df()
# datasets["países_con_region"].shape = (38, 9)
# datasets["renta_2025"].shape = (10, 2)  -- 10 geografías con datos 2025
```

 [DuckDB SELECT](#) · [DuckDB read_csv_auto](#) · [DuckDB Python API](#)

Sector 2 — Transformación de Values (Básico)

Grupo A — Numpy · Pandas · CSV

2.1A) Aritmética y estadística básica con pandas + numpy

```
# [SINTAXIS GENÉRICA] Operaciones vectorizadas sobre columnas y arrays
# pandas: mean, median, std, sum, min, max operan sobre Series
# numpy: np.mean, np.median, np.std operan sobre ndarray (más rápido en arrays grandes)
df["col_c"].mean()           # media de una columna
df["col_c"].median()         # mediana
df["col_c"].std()            # desviación estándar
np.mean(arr)                 # media numpy (útil para arrays multidimensionales)
np.percentile(arr, 75)       # percentil 75

# [EJEMPLO APLICADO] Estadísticas de renta disponible bruta (2025)
renta_2025 = renta[renta["TIME_PERIOD"] == 2025]
renta_2025["OBS_VALUE"].mean()   # 30581.60
renta_2025["OBS_VALUE"].median() # 30616.00
np.percentile(renta_2025["OBS_VALUE"].values, 25) # percentil 25 de renta
```

```
# [SINTAXIS GENÉRICA] Operaciones vectorizadas sobre columnas y arrays
# pandas: mean, median, std, sum, min, max operan sobre Series
# numpy: np.mean, np.median, np.std operan sobre ndarray (más rápido en arrays grandes)
df["col_c"].mean()           # media de una columna
df["col_c"].median()         # mediana
df["col_c"].std()            # desviación estándar
np.mean(arr)                 # media numpy (útil para arrays multidimensionales)
np.percentile(arr, 75)       # percentil 75

# [EJEMPLO APLICADO] Estadísticas de renta disponible bruta (2025)
renta_2025 = renta[renta["TIME_PERIOD"] == 2025]
renta_2025["OBS_VALUE"].mean()   # 30581.60
renta_2025["OBS_VALUE"].median() # 30616.00
np.percentile(renta_2025["OBS_VALUE"].values, 25) # percentil 25 de renta
```

2.2A) Operaciones con strings categóricas: limpieza y combinación

```
# [SINTAXIS GENÉRICA] Métodos vectorizados de strings en pandas (accesor .str)
# split divide por separador; replace usa regex; strip elimina espacios
# cat concatena columnas string con separador
df["col"].str.split(" & ")           # divide string en lista
df["col"].str.replace(r"\\d+", "", regex=True) # elimina patrones tipo [6], [18]
df["col"].str.strip()                 # elimina espacios iniciales/finales
df["nuevo"] = df["col_a"].str.cat(df["col_b"], sep=" - ") # concatena columnas

# [EJEMPLO APLICADO] Limpieza de idiomas oficiales y split de regiones en EuInfo
eu["idioma_limpio"] = eu["Official language(s)"].str.replace(r"\\d+", "", regex=True).str.strip()
```

Grupo B — PostgreSQL · SQLAlchemy · DuckDB · CSV

2.1B) First filter intermedio: aritmética y agregación en la lectura

```
-- [SINTAXIS GENÉRICA] Calcular estadísticas ANTES de traer datos al notebook
-- AVG, MEDIAN, STDDEV, SUM son funciones de agregación que reducen filas
-- GROUP BY particiona el cálculo por categoría
SELECT categoria, AVG(numero) AS media, MEDIAN(numero) AS mediana
FROM read_csv_auto('archivo.csv')
WHERE condicion = 'filtro'
GROUP BY categoria;

-- [EJEMPLO APLICADO] Media y mediana de renta por país en 2025
SELECT geo, AVG(OBS_VALUE) AS media_renta, MEDIAN(OBS_VALUE) AS mediana_renta
FROM read_csv_auto('RentaDisponibleBrutaHogares.csv')
WHERE TIME_PERIOD = 2025
GROUP BY geo;
```

2.2B) First filter con strings categóricas: limpieza en SQL

```
-- [SINTAXIS GENÉRICA] Funciones de string en DuckDB: string_split, regexp_replace, trim
-- Permiten limpiar y transformar texto sin cargar el CSV en memoria local
SELECT id,
       string_split(col, ' & ') AS lista,
       trim(regexp_replace(col, '\\d+', '')) AS limpio
FROM read_csv_auto('archivo.csv');

-- [EJEMPLO APLICADO] Limpieza de idiomas y split de regiones en EuInfo.csv
SELECT "Código",
       string_split("Regions", ' & ') AS regiones_arr,
       trim(regexp_replace("Official language(s)", '\\d+', '')) AS idioma_oficial
FROM read_csv_auto('EuInfo.csv');
```

 [DuckDB funciones de agregación](#) · [DuckDB funciones de string](#)

```
eu["regiones_list"] = eu["Regions"].str.split(r"\s*&\s*", regex=True)
# Ejemplo ES: ['Iberian Peninsula', 'Atlantic Europe', 'Macaronesia', 'Mediterranean countries']
```

 https://pandas.pydata.org/docs/user_guide/text.html · <https://numpy.org/doc/stable/reference/generated/numpy.mean.html> · <https://numpy.org/doc/stable/reference/generated/numpy.percentile.html>

Sector 3 — Data Cleaning

Grupo A — Numpy · Pandas · CSV

3.1A) Detectar valores erróneos y nulos

```
# [SINTAXIS GENÉRICA] Detección de datos faltantes y tipos inconsistentes
# isna() genera máscara booleana; sum() agrega para contar por columna
# info() resume tipos y memoria; duplicated() detecta filas repetidas
df.isna().sum()          # conteo de nulos por columna
df.info()               # resumen estructural + uso de memoria
df.duplicated().sum()    # número de filas duplicadas
df.duplicated(subset=["col_clave"]).sum()  # duplicados en subset de columnas

# [EJEMPLO APLICADO] Diagnóstico de calidad en EuInfo
eu.isna().sum()          # National language(s): 30, Regional: 27, Widely spoken: 18
renta.isna().sum()       # OBS_FLAG: 315 nulos, CONF_STATUS: 379 nulos
```

3.2A) Eliminar o sustituir valores erróneos

```
# [SINTAXIS GENÉRICA] dropna elimina filas/columnas con nulos; fillna rellena
# drop_duplicates elimina repeticiones; inplace=True modifica el objeto original
df_clean = df.dropna(subset=["col_obligatoria"])  # elimina filas donde col es nula
df["col"] = df["col"].fillna("valor_default")    # sustituye nulos por valor
df.drop_duplicates(subset=["id"], inplace=True)   # elimina duplicados por id

# [EJEMPLO APLICADO] Limpieza de EuInfo y Renta
eu_clean = eu.dropna(subset=["Regions"])         # 39 -> 38 filas (elimina BY)
eu["Widely spoken"] = eu["Widely spoken"].fillna("N/A")
renta_clean = renta.dropna(subset=["OBS_VALUE"])  # conserva solo registros con valor
```

 [Missing data en pandas · https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html](https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html) · <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html>

Grupo B — PostgreSQL · SQLAlchemy · DuckDB · CSV

3.1B) Detectar/omitir nulos en la propia consulta (First Filter de cleaning)

```
-- [SINTAXIS GENÉRICA] Filtrar nulos ANTES de cargar datos al entorno local
-- IS NULL detecta; IS NOT NULL omite; ambos se ejecutan en el motor de DuckDB
SELECT * FROM read_csv_auto('archivo.csv') WHERE col IS NULL;      -- detectar nulos
SELECT * FROM read_csv_auto('archivo.csv') WHERE col IS NOT NULL;  -- omitir nulos

-- [EJEMPLO APLICADO] Filtrar países sin región y registros de renta incompletos
SELECT "Código", "Nombre del país" FROM read_csv_auto('EuInfo.csv') WHERE "Regions" IS NULL;  -- Belarús (BY)
SELECT * FROM read_csv_auto('RentaDisponibleBrutaHogares.csv') WHERE OBS_VALUE IS NOT NULL AND TIME_PERIOD = 2025;
```

3.2B) Sustituir valores erróneos en la lectura (COALESCE)

```
-- [SINTAXIS GENÉRICA] COALESCE devuelve el primer valor no nulo de la lista
-- Equivalente a fillna pero ejecutado en el motor de base de datos
SELECT id, COALESCE(col_nula, 'default') AS col_limpia
FROM read_csv_auto('archivo.csv');

-- [EJEMPLO APLICADO] Sustituir idiomas ampliamente hablados no informados
SELECT "Código", "Nombre del país",
       COALESCE("Widely spoken", 'No especificado') AS widely_spoken
FROM read_csv_auto('EuInfo.csv');
```

 [DuckDB y valores NULL](#) · [DuckDB COALESCE](#)

Sector 4 — Condicionantes Vectorizados

Grupo A — Numpy · Pandas · CSV

4.1A) Máscaras booleanas compuestas con pandas + numpy

```
# [SINTAXIS GENÉRICA] Métodos vectorizados de strings en pandas (accesor .str)
# split divide por separador; replace usa regex; strip elimina espacios
# cat concatena columnas string con separador
df["col"].str.split(" & ")          # divide string en lista
df["col"].str.replace(r"[\d+\\]", "", regex=True)  # elimina patrones tipo [6], [18]
df["col"].str.strip()               # elimina espacios iniciales/finales
df["nuevo"] = df["col_a"].str.cat(df["col_b"], sep=" - ")  # concatena columnas

# [EJEMPLO APLICADO] Limpieza de idiomas oficiales y split de regiones en EuInfo
eu["idioma_limpio"] = eu["Official language(s)"].str.replace(r"[\d+\\]", "", regex=True).str.strip()
eu["regiones_list"] = eu["Regions"].str.split(r"\s*&\s*", regex=True)
# Ejemplo ES: ['Iberian Peninsula', 'Atlantic Europe', 'Macaronesia', 'Mediterranean countries']

# [SINTAXIS GENÉRICA] Filtrado vectorizado sin bucles explícitos
# & (AND), | (OR), ~ (NOT) operan elemento a elemento sobre Series booleanas
# np.where selecciona entre dos valores según condición (vectorizado)
# np.select evalúa múltiples condiciones secuenciales
mask = (df["col_a"] > umbral) & (df["col_b"].str.contains("patron", na=False))
df.loc[mask, ["col_x", "col_y"]]    # indexado por máscara compuesta
df["cat"] = np.where(df["val"] > 100, "alto", "bajo")  # clasificación binaria
df["cat"] = np.select([cond1, cond2, cond3], ["A", "B", "C"], default="Z")
```

Grupo B — PostgreSQL · SQLAlchemy · DuckDB · CSV

4.1B) Máscaras compuestas en SQL (First Filter avanzado)

```
-- [SINTAXIS GENÉRICA] Condicionantes múltiples en WHERE con JOIN entre fuentes
-- LIKE '%patron%' busca subcadena; AND/OR combinan condiciones
SELECT t1.col_a, t2.col_num
FROM read_csv_auto('a.csv') t1
JOIN read_csv_auto('b.csv') t2 ON t1.id = t2.id
WHERE t2.col_num > umbral
  AND t1.col_txt LIKE '%patron%';

-- [EJEMPLO APLICADO] Países Alpine con renta > 30000 en 2025 (JOIN en DuckDB)
SELECT e."Nombre del país", r.OBS_VALUE, e."Regions"
FROM read_csv_auto('EuInfo.csv') e
JOIN read_csv_auto('RentaDisponibleBrutaHogares.csv') r ON e."Código" = r.geo
WHERE r.TIME_PERIOD = 2025
  AND r.OBS_VALUE > 30000
  AND e."Regions" LIKE '%Alpine%';
-- Resultado: Alemania | 38462 | ...'Alpine countries'...
```

 [DuckDB JOIN](#) · [DuckDB LIKE](#)


```
# [EJEMPLO APLICADO] Países Alpine con renta > 30000 (merge EuInfo + Renta 2025)
renta_2025 = renta[renta["TIME_PERIOD"] == 2025][["geo", "OBS_VALUE"]]
merged = eu.merge(renta_2025, left_on="Código", right_on="geo", how="inner")
mask = (merged["OBS_VALUE"] > 30000) & (merged["Regions"].str.contains("Alpine", na=False))
merged.loc[mask, ["Nombre del país", "OBS_VALUE", "Regions"]]
# Resultado: Alemania, 38462, contiene 'Alpine countries'
merged["nivel_np"] = np.where(merged["OBS_VALUE"] > 35000, "alto",
                             np.where(merged["OBS_VALUE"] > 25000, "medio", "bajo"))
```

 [Indexado booleano en pandas · https://numpy.org/doc/stable/reference/generated/numpy.where.html](https://numpy.org/doc/stable/reference/generated/numpy.where.html) · <https://numpy.org/doc/stable/reference/generated/numpy.select.html>

Sector 5 — Agrupación Avanzada + One-Hot Encoding

Grupo A — Numpy · Pandas · CSV

5.1A) apply / lambda + groupby por columna

Python

```
# [SINTAXIS GENÉRICA] apply aplica función elemento a elemento (o por fila/columna)
# lambda define función anónima inline; groupby agrupa para operaciones por segmento
# agg permite múltiples agregaciones simultáneas
df["cat"] = df["val"].apply(lambda x: "alto" if x > 100 else ("medio" if x > 60 else "bajo"))
df.groupby("cat")["val"].agg(["mean", "count", "std"]) # estadísticas por grupo

# [EJEMPLO APLICADO] Clasificar renta 2025 en niveles y agregar
merged["nivel"] = merged["OBS_VALUE"].apply(
    lambda x: "alto" if x > 35000 else ("medio" if x > 25000 else "bajo")
)
merged.groupby("nivel")["OBS_VALUE"].agg(["mean", "count"])
# alto: media 37319 (2 países) | medio: media 29674.5 (4 países) | bajo: media 22622 (1 país)
```

5.2A) One-Hot Encoding

```
# [SINTAXIS GENÉRICA] pd.get_dummies convierte categórica a variables dummy (0/1)
# prefix añade prefijo a las nuevas columnas; drop_first evita colinealidad
dummies = pd.get_dummies(df["categoria"], prefix="cat") # one-hot con pandas
df = pd.concat([df, dummies], axis=1) # añade al DataFrame original

# [EJEMPLO APLICADO] One-hot del nivel de renta clasificado
dummies_nivel = pd.get_dummies(merged["nivel"], prefix="nivel")
merged_ohe = pd.concat([merged, dummies_nivel], axis=1)
# Genera: nivel_alto, nivel_medio, nivel_bajo (0/1)
print(dummies_nivel.sum()) # nivel_alto: 2, nivel_medio: 4, nivel_bajo: 1
```

```
# [SINTAXIS GENÉRICA] pd.get_dummies convierte categórica a variables dummy (0/1)
# prefix añade prefijo a las nuevas columnas; drop_first evita colinealidad
dummies = pd.get_dummies(df["categoria"], prefix="cat") # one-hot con pandas
df = pd.concat([df, dummies], axis=1) # añade al DataFrame original

# [EJEMPLO APLICADO] One-hot del nivel de renta clasificado
dummies_nivel = pd.get_dummies(merged["nivel"], prefix="nivel")
merged_ohe = pd.concat([merged, dummies_nivel], axis=1)
# Genera: nivel_alto, nivel_medio, nivel_bajo (0/1)
print(dummies_nivel.sum()) # nivel_alto: 2, nivel_medio: 4, nivel_bajo: 1
```

 <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.apply.html> · <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html> · https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html

Grupo B — PostgreSQL · SQLAlchemy · DuckDB · CSV

5.1B) Condicionantes compuestos + agrupación por headers exactos (SQL)

```
-- [SINTAXIS GENÉRICA] CASE WHEN evalúa condiciones secuenciales (equivalente a apply/lambda)
-- GROUP BY agrupa por múltiples columnas; COUNT(*) y AVG son agregaciones por grupo
SELECT grupo,
       CASE WHEN val > 100 THEN 'alto'
            WHEN val > 60  THEN 'medio'
            ELSE 'bajo' END AS nivel,
       COUNT(*) AS n, AVG(val) AS media
FROM tabla
WHERE periodo = 2025
GROUP BY grupo, nivel;

-- [EJEMPLO APLICADO] Clasificar renta 2025 y agrupar por región (JOIN + CASE)
SELECT e."Regions",
       CASE WHEN r.OBS_VALUE > 35000 THEN 'alto'
            WHEN r.OBS_VALUE > 25000 THEN 'medio'
            ELSE 'bajo' END AS nivel,
       COUNT(*) AS n_paises, AVG(r.OBS_VALUE) AS media_renta
FROM read_csv_auto('EuInfo.csv') e
JOIN read_csv_auto('RentaDisponibleBrutaHogares.csv') r ON e."Código" = r.geo
WHERE r.TIME_PERIOD = 2025
GROUP BY e."Regions", nivel;
```

5.2B) One-Hot Encoding vía SQL (columnas condicionales + PIVOT)

```
-- [SINTAXIS GENÉRICA] One-hot manual con CASE WHEN por cada categoría
-- Alternativa nativa: PIVOT (DuckDB) rota filas a columnas automáticamente
SELECT id,
       CASE WHEN nivel = 'alto' THEN 1 ELSE 0 END AS nivel_alto,
       CASE WHEN nivel = 'medio' THEN 1 ELSE 0 END AS nivel_medio,
       CASE WHEN nivel = 'bajo' THEN 1 ELSE 0 END AS nivel_bajo
FROM tabla_clasificada;

-- PIVOT nativo DuckDB (más compacto)
PIVOT tabla_clasificada ON nivel USING COUNT(*);

-- [EJEMPLO APLICADO] Exportar resultado clasificado a PostgreSQL vía SQLAlchemy
merged_ohe.to_sql("paises_renta_clasificada", engine, if_exists="replace", index=False)
# Carga el DataFrame final (con one-hot encoding) a PostgreSQL, cerrando el flujo DataScientist
```

 https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html · [DuckDB PIVOT](#) · https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_sql.html

Generado a partir de este Markdown — [edítalo](#) y vuelve a generar el PDF para versiones actualizadas.
Ejemplos validados contra *EuInfo.csv* (39 países, 9 columnas) y *RentaDisponibleBrutaHogares.csv* (379 registros, 34 geografías, series 2014-2025).

plain

````plain